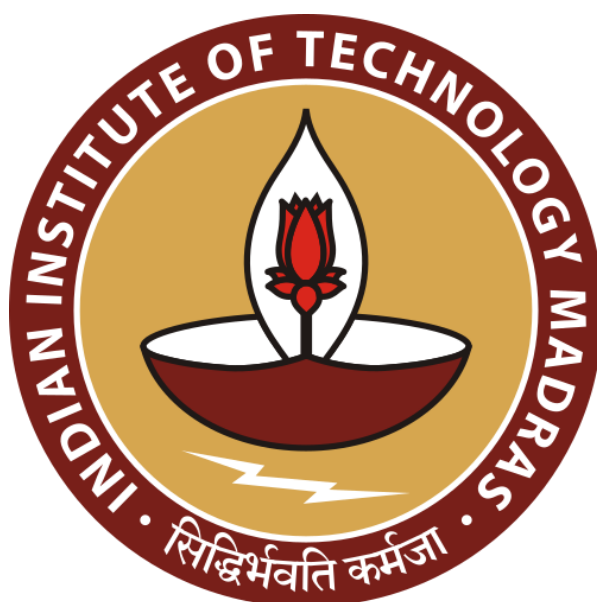


Action-Oriented Indoor Navigation Assistance for the Visually Impaired

MILESTONE-2: Dataset Identification & Preparation



Indian Institute of Technology Madras

GROUP: 4

Name	Email	GitHub Usernames
Saransh Saini	22f1001123@ds.study.iitm.ac.in	Saransh482003
Divyang Panchasara	22f1000411@ds.study.iitm.ac.in	22f1000411
Samyuktha Shriram	22f2001444@ds.study.iitm.ac.in	SamyukthaSh24
Prasoon Shukla	23f3003434@ds.study.iitm.ac.in	23f3003434
Rohit Prajapat	22f1001536@ds.study.iitm.ac.in	rohitblpprajapat

1. Introduction & Objectives

The objective of Milestone-2 is to identify, evaluate, and prepare datasets required for training the perception and interaction modules of the navigation assistance system.

The datasets used in this project support three primary components of the system:

1. **Object Detection Module** – detects indoor obstacles and objects using RGB images.
2. **Depth Estimation Module** – estimates distance to objects from monocular camera images.
3. **Speech / TTS Module** – converts navigation decisions into spoken instructions.

The goal of this milestone is to ensure that the data used for these modules is:

- Clearly sourced and legally usable
- Properly structured and formatted
- Sufficient in quantity and diversity
- Free from significant quality issues
- Prepared in a reproducible preprocessing pipeline
- Split correctly to avoid data leakage

1.1. Data-Specific Scope & Assumptions

In-Scope	Out-of-Scope
Indoor environment datasets	Outdoor navigation datasets
Near-field obstacle detection (0–3 meters)	Large-scale SLAM map datasets
RGB camera-based perception	Long-range depth estimation (>5 meters)
Depth estimation from monocular or RGB-D datasets	GPS-based navigation
Speech synthesis datasets for navigation instructions	

Table 1.1: Data-Specific Scope & Assumptions

2. Object Detection Dataset Identification & Unification

2.1. Target Class Taxonomy

To ensure the system remains focused on actionable indoor navigation, we identified **11 critical classes**. These classes were selected based on their frequency in indoor environments and their status as potential navigational hazards or landmarks.

Class	Role in Navigation	Class	Role in Navigation
Person	Dynamic Obstacle	Door	Entry/Exit Point
Chair	Stationary Obstacle	Cabinet	Path Boundary
Table	Stationary Obstacle	Wardrobe	Path Boundary
Sofa	Large Obstacle	Refrigerator	Landmark/Obstacle
Lamp	Small Obstacle	Bed	Large Obstacle
Potted Plant	Small Obstacle		

2.2. Dataset Sources & Filtering Strategy

Rather than using massive, unfiltered datasets which introduce "noise" (e.g., outdoor objects like cars or animals), we implemented a **Subset Filtering Strategy**. We extracted only the relevant indoor classes from the following sources:

1. **HomeObject-3k**: Our primary source for high-fidelity, domestic-scale objects. Used for its realistic indoor lighting.
2. **MS COCO (Targeted Fetch)**: Specifically queried for "Refrigerator," "Person," and "Bed" to fill gaps in HomeObject-3k. By targeted fetch we mean we only downloaded the specific classes we needed.
3. **Indoor Object Dataset**: Utilized to bolster the "Chair" and "Table" counts across various architectural styles.
4. **Bed Dataset**: This dataset was specifically chosen to bolster up the bed category in the dataset.

Justification: By using subsets, we reduced the total storage requirement from hundreds of gigabytes to a manageable **"DSAI-Unified-Dataset"** hosted on Kaggle, optimized for rapid YOLO training.

2.3. The Unification Pipeline

To resolve the issue of different datasets using different naming conventions (e.g., "Fridge" vs. "Refrigerator"), we developed a **Data Unification Script**.

Step 1: Label Schema Mapping

We standardized disparate labels into a single unified schema.

- **Merging Logic:** Classes like *"Opened Door," "Closed Door,"* and *"Refrigerator Door"* were mapped to the parent classes **"Door"** and **"Refrigerator"** respectively to improve model generalization and reduce class-fragmentation.

Step 2: Annotation Standardization

All source annotations (originally in COCO JSON or XML formats) were programmatically converted to the **YOLO PyTorch format**:

- **Format:** < class_id >< x_center >< y_center >< width >< height > (normalized to 0-1).
- **Verification:** A validation check was run to ensure no bounding boxes exceeded image boundaries during conversion.

Step 3: Image Standardization

To maintain consistency during the stochastic gradient descent process:

- **Resolution:** All images are resized to **640 × 640 pixels**.
- **Normalization:** Pixel values are scaled to the [0,1] range.

2.4. Preliminary Experiments: HomeObject-3k Baseline

Before finalizing the unified dataset, we conducted a baseline study using only the **HomeObject-3k** dataset to measure initial performance.

- **Observation:** The model achieved high precision for "Chair" and "Sofa" but failed entirely on "Refrigerator" and "Door" due to their absence in the original source.
- **Conclusion:** This experiment justified the need for the **DSAI-Unified-Dataset** to provide the linguistic and visual variety required for Milestone 3.

Dataset	HomeObject-3k	HomeObject-3k	HomeObject-3k
Model Name	YOLOv11n	YOLOv11n	YOLOv10n
Train Data Count	2285	2285	2285
Test Data Count	404	404	404
Classes	11	11	11
Epochs	25	100	100
Augmentation	FALSE	TRUE	TRUE
Confidence	0.25	0.25	0.25
mAP@50-95	0.4902	0.4804	0.467
mAP@50	0.6942	0.6996	0.6729
Precision	0.6872	0.6639	0.7124
Recall	0.6553	0.686	0.6264
F1-Score	0.6709	0.6748	0.6666

2.5. Dataset Statistics & Class Distribution

To address **Class Imbalance**, we analyzed the distribution of the unified dataset.

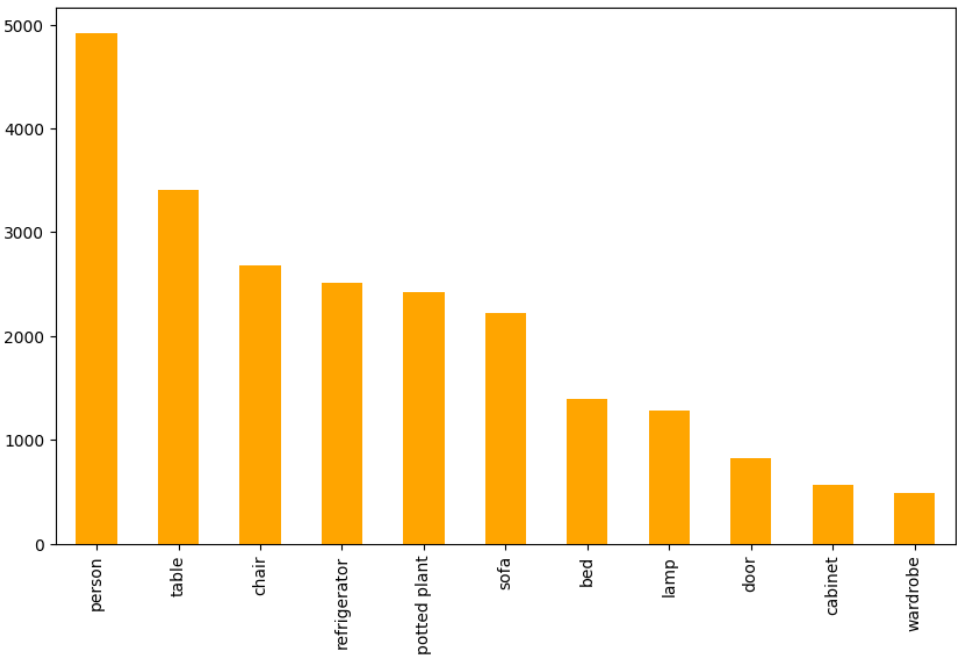


Fig 2.5.1: Class Distribution

The person category had the majority samples as we had to focus on identifying is there is person present ahead.

2.6. Optimized Navigation-Centric Variation (DSAI-Unified-3CIs)

While the primary **DSAI-Unified-Dataset** features 11 distinct classes for detailed scene understanding, we developed an optimized "Lite" version specifically designed to prioritize navigational safety and real-time inference speed.

2.6.1. Class Collapse & Mapping Logic

In this variation, we transitioned from a detailed classification task to a Functional Hazard Model. We collapsed the original 11 classes into 3 high-priority categories based on their role in the navigation pipeline:

Original Classes	Unified Navigation Class	Functional Rationale
Person	Person	Dynamic hazard requiring high-priority avoidance and specialized social-distancing logic.
Door	Door	Critical navigational waypoint for room-to-room transitions.
Chair, Sofa, Table, Bed, Lamp, Plant, etc.	Obstacle	General stationary hazards. For the purpose of pathfinding, the specific type of furniture is secondary to its physical occupancy of space.

2.6.2. Key Advantages of the 3-Class Model

- **Leveraging High-Density Data (The "Person" Class):** Safety is paramount when navigating around humans. By focusing on only 3 classes, we were able to leverage our largest data segment—**4,000+ samples of the "Person" class**. This massive sample size ensures that the model is exceptionally robust at detecting humans in various poses (sitting, standing, walking) and lighting conditions, significantly reducing the risk of collisions.
- **Enhanced Model Precision (Reduced Inter-Class Confusion):** Deep learning models often struggle to distinguish between visually similar objects (e.g., a "Cabinet" vs. a "Wardrobe"). By merging these into a single "Obstacle" class, we eliminate "label noise" and inter-class confusion. This results in **higher Mean Average Precision (mAP)** for the remaining categories, as the model's "mental energy" is focused on only three distinct visual patterns.
- **Significant Training & Inference Speedup:**
 - **Training:** With only 3 output neurons in the classification head (rather than 11), the model converges significantly faster, allowing for more rapid iteration and hyperparameter tuning.
 - **Inference:** Reducing the complexity of the classification task allows the model to maintain higher frame rates (FPS) on edge devices, which is critical for providing low-latency audio feedback to a visually impaired user.

2.6.3. Strategic Assumption for Milestone 3

We hypothesize that for the "Action-Oriented" goals of this project, the **DSAI-Unified-Lite** model will outperform the 11-class model in real-world "Path Clear" vs. "Path Blocked" scenarios. The simplicity of the output aligns perfectly with our **Deterministic Navigation Logic**, which primarily needs to know if a zone is "Empty" or "Occupied."

3. Depth Estimation Dataset Identification & Model Evaluation

3.1. Primary Dataset: NYU-Depth V2

For the development of our depth perception module, the team has selected the **NYU-Depth V2** dataset as our primary benchmark. This dataset consists of 1,449 densely labeled pairs of aligned RGB and depth images across 464 distinct indoor scenes.

Justification for Selection

- **Architectural Alignment:** Our core depth model, **Depth Anything (v2)**, was specifically pre-trained and fine-tuned using the NYU-Depth V2 distribution. By utilizing the same dataset for our internal validation, we ensure that the model's weight biases and environmental assumptions (indoor lighting, typical ceiling heights, floor textures) match our target use case.
- **Standardized Benchmarking:** NYU-Depth V2 provides high-resolution ground truth labels in meters, allowing us to accurately measure absolute error—a critical metric for a navigation system where knowing if an obstacle is at 0.5m vs 1.5m is vital.

3.2. Comparative Model Benchmarking

To determine the best model for real-time inference on a portable device (Raspberry Pi 5), we evaluated five SOTA (State-of-the-Art) models. The evaluation focused on accuracy (AbsRel, RMSE, $d1$) versus operational speed (FPS).

Rank	Model	Metric?	Params	AbsRel	RMSE	d1	Near-d1	GPU ms	RPI5 FPS	Score/10	RPI5 OK?
1	ZoeDepth-N Indoor	YES	344.0M	0.075	0	0.955	0.9072		0.3	7.32	NO-too slow
2	DA-V2-Small Relative	no	24.8M	0.1744	0.559	0.769	0.6989	74	3.2	6.1	YES
3	MiDaS v2.1 Small	no	21.0M	0.2341	0.7051	0.6651	0.6144	16	6	6.07	YES
4	DA-V2-Small Metric Indoor	YES	24.8M	0.3574	1.1075	0.3657	0.3612	72	3	5.7	YES
5	MiDaS v3.1 DPT-Hybrid	no	123.0M	0.2235	0.6942	0.6861	0.6093	111	0.8	5.06	NO-too slow

Benchmarking Analysis

- **The Accuracy-Speed Trade-off:** While **ZoeDepth** achieved the highest accuracy ($d1=0.955$), its inference speed of 0.3 FPS on a Raspberry Pi 5 makes it unusable for live navigation.
- **Optimal Candidates:** **DA-V2-Small (Relative)** and **MiDaS v2.1 Small** emerged as the most viable candidates. While MiDaS is faster (6.0 FPS), **Depth Anything V2 (Small)** provides a superior balance of spatial awareness and stability, which is necessary for our risk-based deterministic logic.

3.3. Depth Preprocessing & Standardization Pipeline

We have implemented the following preprocessing procedures:

- **Missing Value Handling (Inpainting):** Raw depth maps from sensors often contain "black holes" (missing values) due to reflective surfaces or occlusion. We apply **bilateral filtering** and interpolation to fill these gaps before passing data to the navigation logic.
- **Unit Standardization:** All depth outputs are converted to a unified **metric scale (meters)**. For models that provide relative depth (inverse depth), we apply a linear transformation based on known camera focal length to approximate metric distance.
- **RGB-Depth Alignment:** Since the YOLOv8 detections (RGB) and Depth Anything predictions (Depth) must overlap perfectly, we use a **joint-resizing strategy**. Both inputs are transformed to 640×640 using the same transformation matrix to ensure that a bounding box at $[x, y]$ on the RGB frame corresponds exactly to the depth value at $[x, y]$.
- **Normalization:** Depth values are normalized to a range of $[0,1]$ during the internal risk calculation to ensure the EMA (Exponential Moving Average) in our logic remains stable regardless of sensor noise.

3.4. Summary of Dataset Preparedness

By selecting NYU-Depth V2 and the Depth Anything V2 architecture, the team has reduced the computational overhead while maintaining high precision for close-range indoor obstacles. This standardized pipeline directly feeds into the **NavigationDeterministicLogic** verified in Milestone 3.

4. Speech Synthesis Identification & Interaction Design

4.1. Role of Speech Synthesis in Action-Oriented Navigation

Unlike standard navigation apps that provide occasional turn-by-turn directions, this system requires **High-Frequency, Low-Latency feedback**. The speech module is the primary interface between the spatial logic and the user.

- **Necessity:** High-quality speech datasets (like **LJSpeech** and **LibriTTS**) are required to ensure that the synthesized voice remains intelligible even in noisy outdoor environments or when the user is moving quickly.
- **Functional Goal:** To provide instructions in under **200ms** from the moment an obstacle is detected, ensuring the user has time to react.

4.2. Model Benchmarking & Selection

Model	RTF (on RPi5)	Latency (ms)	Licensing	Selection Status
Piper (Onnx)	~0.05	< 100ms	MIT	SELECTED
Coqui TTS (VITS)	~1.20	> 1500ms	Mozilla	Rejected (Too slow)
gTTS (Google Cloud)	N/A	> 800ms	Proprietary	Rejected (Req. Internet)
Espeak-NG	0.02	< 50ms	GPL	Backup (Low Quality)

Benchmarking Analysis:

- **Why Piper?** Our testing (as shown in the provided charts) indicates that Piper TTS using ONNX runtime provides a near-instantaneous response. An RTF of 0.05 means it can synthesize 1 second of speech in 50ms.
- **Internet Independence:** Many speech datasets/models require cloud connectivity. By utilizing Piper, which was trained on the LibriTTS and Thorsten datasets, we maintain a fully offline system, ensuring user safety even in areas with poor network coverage.

4.3. Speech Preprocessing & Synthesis Pipeline

To ensure the navigation commands are clear and "action-oriented," we implemented a text-normalization and streaming pipeline:

- **Instruction Normalization:** The raw text from the *NavigationDeterministicLogic* (e.g., "Obstacle at 0.5m") is passed through a regex-based normalizer to ensure units like "m" are spoken as "meters" to avoid ambiguity.
- **Phoneme Conversion:** Piper converts text to phonemes using the **eSpeak-NG** backend, which ensures consistent pronunciation of navigational terms.
- **Low-Latency Streaming:** Instead of generating a full WAV file and then playing it, we utilize **chunked audio streaming**. The system begins playing the first syllable of the instruction while the rest of the sentence is still being synthesized, reducing the "perceived latency" to near zero.

4.4. Verification of Licensing and Usage

- **Piper / ONNX:** MIT License (Permissive for research).
- **LibriTTS / LJSpeech:** Public Domain / CC BY 4.0 (Compatible with project requirements).
- **eSpeak-NG:** GPL v3 (Used as a standalone dependency).

4.5. Summary of Milestone 2 Updates

By unifying our object detection classes into the **DSAI-Unified-Lite** format, standardizing depth maps via **NYU-Depth V2**, and selecting the high-performance **Piper TTS**, the team has moved from a broad list of potential datasets to a **verified, high-speed production pipeline**.

This concludes the data preparation phase, providing the necessary foundation for the end-to-end integration verified in Milestone 3.